

Lab 3: The Format Detective's Casebook

Overview

This lab has four tasks. Task A builds a programmatic inventory of spatial data files. Task B identifies concrete behavioral differences between formats, with code evidence for each. Task C asks you to pick one format and standardize the project on it, justifying your choice. Note, there's no single correct answer here, only a defensible one. Task D closes with a short recommendation memo.

Download the template notebook, `lab03_template.ipynb`, before you start. It has one markdown header and one seeded code cell per task below.

Data: all of it already in the repository at `/data/part1/formats_zoo/`. Four items are included: `counties.shp` (with its usual sidecars - `.shx`, `.dbf`, `.prj`, `.cpg`), `counties.gpkg`, `counties.parquet`, and `counties_attrs.csv` (just the table). Save your notebook in the same folder.

Setup

Set the workspace and turn the overwrite flag on.

```
arcpy.env.workspace = "YOURPATHHERE/data/part1/formats_zoo/"
arcpy.env.overwriteOutput = True
```

This lab introduces two tools you haven't seen before:

- `arcpy.management.CreateSQLiteDatabase(out_database_name, spatial_type)` — creates an empty SQLite database file. With `spatial_type` set to `"GEOPACKAGE"`, the file it creates is an empty GeoPackage. This is required because arcpy's export tools can write a layer *into* a GeoPackage, but they won't create the `.gpkg` container itself.
- `arcpy.management.AddField(in_table, field_name, field_type)` — adds a new, empty field of the given type to an existing feature class or table.

Before we move on to Task A, we're going to create a new GeoPackage. Accordingly your inventory will contain five containers (instead of four).

This requires two steps: 1. Create the empty GeoPackage first 2. Export the shapefile into it as a layer named `counties`

```
arcpy.management.CreateSqliteDatabase("counties_pro.gpkg", "GEOPACKAGE")
arcpy.conversion.ExportFeatures("counties.shp", "counties_pro.gpkg/counties")
```

Order matters — the container has to be there before a layer can be written into it.

You now have five items in the workspace: `counties.shp`, `counties.gpkg`, `counties.parquet`, `counties_attrs.csv`, and `counties_pro.gpkg/counties`. Each contains the same 88 counties.

Task A: Programmatic inventory (19 points)

Build a table with one row per item in the case file and four columns: **format**, **size on disk**, **field count**, and **CRS**.

EXAMPLE: Below is an *EXAMPLE* code block showing how to create a single row in the inventory table with a stand-in file (note, this is NOT one of your five - the numbers below are illustrative only, so this doesn't hand you an answer for the real case file):

```
import os

file_path = "C:/YOURPATH/formats_zoo/sample_layer.gpkg"
layer_path = "C:/YOURPATH/formats_zoo/sample_layer.gpkg/main.sample_layer"
fmt = "GeoPackage"
size_bytes = os.path.getsize(file_path)

desc = arcpy.da.Describe(layer_path)
field_count = len(arcpy.ListFields(layer_path))
crs = desc["spatialReference"].name

print(f"{fmt:<12} {size_bytes:>10} bytes {field_count:>3} fields {crs}")
```

Note the GeoPackage row needs two paths: 1. the `.gpkg` file for `os.path.getsize()` 2. the `main.-prefixed layer` inside it for `Describe` and `ListFields`

Every row in your real inventory follows this same general design.

Because the five items are of different types (multi-file bundle, two single `.gpkg` files, a single `.parquet` file, and a plain CSV with no geometry at all), a `ListFeatureClasses()` call will not discover all five.

Every value in the table itself (size, field count, CRS) has to come from code, not from typing in a number you read off a properties dialog.

A few things worth knowing before you write the loop:

- **Size on disk** isn't one line of code for every item. A single file (either `.gpkg`, the `.parquet`, the `.csv`) is what `os.path.getsize()` expects. A shapefile isn't a single file. `os.scandir()` loops a folder's entries the same way `os.listdir()` does, but hands back objects with a `.stat()` you can call directly:

```
import os

for entry in os.scandir("C:/YOURPATH/formats_zoo"):
    if entry.is_file():
        print(f"{entry.name:<24} {entry.stat().st_size} bytes")
```

The shapefile is its own challenge: `counties.shp` contains only the geometry. Its real “size on disk” is the sum of every sidecar file sharing the name `counties` (`.shp`, `.shx`, `.dbf`, `.prj`, `.cpg`) which can be calculated by a `os.scandir()` loop, filtered on a matching base name.

Hint: two cases, one decision per item

Before you write the size line for a given item, ask which of two buckets it falls into:

1. **A single file** (`.gpkg`, `.parquet`, `.csv`)? Then `os.path.getsize(path)` is the answer
2. **A multi-file bundle sharing a base name** (the shapefile)? Then loop `os.scandir()` over the folder, keep only entries whose name starts with the base name (`"counties."`), and sum `entry.stat().st_size` across the matches. Watch the trailing dot in `"counties."`

You may need to write one small helper function that takes a path and returns a size handling both cases. You could then call it from your inventory loop, rather than repeating the branching logic five times.

- **Field count and CRS** come from `arcpy.da.Describe()`, the same dictionary-style access from in-class. `counties_attrs.csv` is the odd-ball - it's a table, not a feature class, so it has no coordinate system.

Attempting to read `desc["spatialReference"]` from a plain table raises a `KeyError`, so you'll need to wrap that specific access in `try/except` block and report "N/A" when it fails. After all, you wouldn't want to let a non-spatial item crash the whole loop.

- `counties.parquet` is potentially problematic. Whether `arcpy.da.Describe()` can open a `.parquet` file depends on the ArcGIS Pro version on the machine you're using. If it raises an error for you, catch it, report what `os.path.getsize()` gives you, and note in your table that `Describe` wasn't available for that row on your machine.

Do not hardcode any of the computed values.

Task B: The search for differences (23 points)

An inventory table tells you *what's present* in the directory. This task requires you to go a step further and find *what actually behaves differently* between formats. Here, we're looking for concrete differences, each provided by code.

Two are required. You have everything you need to identify them:

1. The ten-character wall. our in-class discussions and notebooks explained shapefile's 10-character field-name limit, even if we didn't work with it directly. Add a field with a name well over 10 characters to the GeoPackage copy you built in Setup, then export that layer to a fresh shapefile and compare field names on both sides. This will be the one new tool in this lab: `AddField`. Here's an example:

```
arcpy.management.AddField("counties_pro.gpkg/main.counties", "median_household_income", "DOUBLE")
arcpy.management.CopyFeatures("counties_pro.gpkg/main.counties", "trunc_test.shp")

gpkg_fields = [f.name for f in arcpy.ListFields("counties_pro.gpkg/main.counties")]
shp_fields = [f.name for f in arcpy.ListFields("trunc_test.shp")]
```

Print both lists. The GeoPackage field name should come through intact; the shapefile version won't. State, in your own words, what the shapefile version's name actually is and why.

Note the use of a different tool. This step uses `arcpy.management.CopyFeatures`, not the `ExportFeatures` call from Setup. `ExportFeatures` checks every output field name against the output format before it writes anything, and a 23-character name is illegal in a shapefile's dBASE table, so it stops with error 000142 instead of producing a file. `CopyFeatures` does no field mapping; it writes the shapefile, shortens the name to fit, and reports a warning.

2. Two GeoPackages, two different primary-key names. `counties.gpkg` (the one that was already in the folder) and `counties_pro.gpkg` (the one you built in Setup) are both, structurally, the same kind of container (i.e., a single-file SQLite database). Compare their `OIDFieldName` (the field `arcpy` uses as each dataset's unique row identifier) using `arcpy.da.Describe()`:

```
old_desc = arcpy.da.Describe("counties.gpkg/main.counties")
new_desc = arcpy.da.Describe("counties_pro.gpkg/main.counties")

print(f"counties.gpkg: {old_desc['OIDFieldName']}")
print(f"counties_pro.gpkg: {new_desc['OIDFieldName']}")
```

Both paths use the `main.` prefix — Esri addresses a GeoPackage's layers as `main.<layer>`. The two `OIDFieldName` values should not match, even though both files are GeoPackages holding the identical 88 counties. In a sentence, explain why. As a hint, consider the process used to create each file.

Pick a third difference yourself. Maybe file count? Size? Something else?

Task C: Standardize (19 points)

Pick **one** format and make the case that it's the right one to standardize this project on going forward. There's no single correct answer here.

Produce a clean `standardized/` subfolder inside `formats_zoo` containing exactly one canonical copy of the 88 counties, written in your chosen format. If you pick GeoParquet, remember the caveat about version compatibility.

In a markdown cell underneath, write 4-6 sentences justifying your choice: what you're optimizing for (e.g., compatibility, size, editability, standards-compliance, something else), and what you're giving up by not choosing a different format.

Task D: Recommendation memo (10 points)

Write a memo, **300 words or fewer**, recommending a format for each of three separate situations:

- (a) **emailing a finished dataset to a colleague** who may or may not be running Esri software,
- (b) **archiving a 10-million-row dataset** that will mostly be read, rarely edited, and

- (c) **daily hands-on editing inside ArcGIS Pro**, on a project that will grow subtypes and domains over time.

Ground each recommendation in something you actually observed this lab rather than restating week 4's lecture.

Deliverables and submission

Submit through Canvas:

1. Your completed template notebook (`lab03_template.ipynb`), containing Setup's new GeoPackage, Task A's inventory table and the code that produced it, Task B's three differences with code evidence and interpretation, Task C's standardized output and justification, and Task D's memo.

Submit code as code, not screenshots of code

Point breakdown

Component	Points
Task A: Programmatic inventory	19
Task B: The differences hunt	23
Task C: Standardize	19
Task D: Recommendation memo	10
Style and naming	4
Total	75

“Style and naming” covers `snake_case` variable names, f-strings for all formatted output, and derived (not hand-typed) values anywhere the lab asks for something computed.

Late work follows the course late policy.